Node.js & Express RESTful API with Middleware

Request lifecycle, routing, validation, errors, security, and testing

1. The REST resource model

A RESTful API exposes resources through stable URLs and uses HTTP methods to express intent. Prefer nouns such as /users and /orders over action-heavy paths. GET retrieves, POST creates or triggers processing, PUT replaces, PATCH partially updates, and DELETE removes. Correct method semantics improve caching, retries, observability, and client expectations.

```
GET    /api/v1/books
GET    /api/v1/books/42
POST   /api/v1/books
PATCH  /api/v1/books/42
DELETE /api/v1/books/42
```

Design notes: Use plural resource names, predictable identifiers, consistent error bodies, and explicit API versioning where compatibility requires it. Avoid exposing database table names or internal implementation details. Design the response shape as a public contract, not a serialization accident.

Practice: Design exercise: define endpoints for a library with books and loans. Identify which operations are safe and which may be retried without creating duplicates.

2. A minimal Express application

Express is a small web framework layered over Node's HTTP server. Middleware and route handlers run in registration order. Keep application construction separate from the listener so tests can import the app without binding a port.

```
import express from 'express';

const app = express();
app.disable('x-powered-by');
app.use(express.json({ limit: '100kb' }));

app.get('/health', (_req, res) => {
  res.status(200).json({ status: 'ok' });
});

const port = Number(process.env.PORT ?? 3000);
app.listen(port, () => {
  console.log(`Listening on ${port}`);
});
```

Design notes: Use a currently supported Node.js release and a compatible Express version. Set body limits deliberately, load configuration from environment variables, and avoid logging secrets. A health endpoint should report service health without leaking internal topology or credentials.

Practice: Experiment: add a route and test it with curl. Verify the status code and Content-Type rather than only checking the browser output.

Additional review: Implementation lens: isolate the smallest reproducible example, state the expected result, and test it independently before integrating it into a larger application.

3. Middleware as a pipeline

Middleware functions receive request, response, and next. They may inspect or modify the request, end the response, or pass control onward. If middleware neither ends the response nor calls next, the request hangs. Error middleware uses four parameters: (err, req, res, next).

```
function requestTimer(req, res, next) {
  const started = process.hrtime.bigint();
  res.on('finish', () => {
    const elapsedMs = Number(process.hrtime.bigint() - started) / 1e6;
    console.log(req.method, req.path, res.statusCode,
                elapsedMs.toFixed(1) + 'ms');
  });
  next();
}

app.use(requestTimer);
app.use(express.json());
app.use('/api/v1/books', booksRouter);
```

Design notes: Place general instrumentation early, parsers before routes that need parsed bodies, authentication before protected handlers, and not-found/error handlers after routes. Treat middleware order as executable policy. Avoid storing per-request state in module-level mutable variables.

Practice: Debugging drill: deliberately omit next() in a test middleware and observe the hanging request. Then fix the control flow.

4. Routing, parameters, and controllers

Use Router to group related endpoints and keep route declarations readable. Route parameters identify resources; query parameters generally control filtering, sorting, or pagination. Parse and validate both rather than trusting their string values.

```
import { Router } from 'express';

const router = Router();

router.get('/', async (req, res, next) => {
  try {
    const limit = Math.min(Number(req.query.limit ?? 20), 100);
    if (!Number.isInteger(limit) || limit < 1) {
      return res.status(400).json({ error: 'invalid_limit' });
    }
    const books = await repository.list({ limit });
    res.json({ data: books });
  } catch (err) {
    next(err);
  }
});

router.get('/:bookId', getBookById);
export default router;
```

Design notes: Separate transport concerns (HTTP status, parsing, response shape) from domain logic and persistence. A controller should orchestrate work, not become a single giant function. Be explicit about maximum pagination sizes to prevent accidental expensive requests.

Practice: Exercise: add ?author=... and ?limit=... filters. Decide whether an unknown filter should be ignored or rejected, and document that choice.

5. Input validation and normalization

All client input is untrusted, including path parameters, query strings, headers, and JSON bodies. Validate required fields, types, lengths, ranges, enumerated values, and cross-field rules at the boundary. Normalize only where semantics are well-defined, such as trimming a human name.

```
function validateCreateBook(req, res, next) {
  const { title, author, year } = req.body ?? {};
  if (typeof title !== 'string' || title.trim().length < 1 ||
      title.trim().length > 200) {
    return res.status(400).json({
      error: 'invalid_title',
      message: 'title must contain 1–200 characters'
    });
  }
  if (typeof author !== 'string' || author.trim() === '') {
    return res.status(400).json({ error: 'invalid_author' });
  }
  if (year !== undefined &&
      (!Number.isInteger(year) || year < 0 || year > 2100)) {
    return res.status(400).json({ error: 'invalid_year' });
  }
  req.body = { title: title.trim(), author: author.trim(), year };
  next();
}
```

Design notes: Schema validation libraries can centralize validation, but the schema must match business rules. Never use validation as a substitute for authorization. Return useful errors without reflecting secrets or raw stack traces.

Practice: Test missing fields, nulls, wrong types, very long strings, unexpected arrays, boundary values, and extra properties.

Additional review: Review lens: consider edge cases, failure behavior, maintainability, and the assumptions the code makes about its input or execution environment.

6. Status codes and stable error contracts

Status codes communicate the outcome to clients and intermediaries. Use 200 for successful retrieval/update responses, 201 for creation, 204 for success without a body, 400 for malformed or invalid input, 401 for missing/invalid authentication, 403 for insufficient permission, 404 for absent resources, 409 for conflicts, and 500 for unexpected server failures.

```
app.use((req, res) => {
  res.status(404).json({
    error: 'not_found',
    message: 'No route matches this request'
  });
});

app.use((err, req, res, next) => {
  if (res.headersSent) return next(err);

  const status = Number.isInteger(err.status) &&
                 err.status >= 400 && err.status < 500
    ? err.status : 500;

  if (status >= 500) console.error(err);
  res.status(status).json({
    error: status >= 500 ? 'internal_error' : (err.code ?? 'request_error')
  });
});
```

Design notes: Do not send stack traces to clients in production. If an endpoint returns a body, make its content type and schema predictable. Consider request IDs so logs and support reports can be correlated.

Practice: Exercise: define errors for duplicate email, expired token, invalid JSON, and unexpected database outage.

7. Async handlers and failure propagation

Database and network operations are asynchronous. Every rejected promise must reach an error path; otherwise failures can become unhandled rejections or requests can be left incomplete. Express-version behavior differs for rejected promises, so verify the behavior of the version in use and use a wrapper when needed.

```
const asyncRoute = (handler) => (req, res, next) =>
  Promise.resolve(handler(req, res, next)).catch(next);

router.get('/:id', asyncRoute(async (req, res) => {
  const item = await repository.findById(req.params.id);
  if (!item) return res.status(404).json({ error: 'not_found' });
  res.json({ data: item });
}));
```

Design notes: Do not swallow exceptions with empty catch blocks. Distinguish expected domain failures from unexpected infrastructure errors. Timeouts, cancellation, retries, and idempotency are part of robust API design, not afterthoughts.

Practice: Inject a repository that rejects its promise and verify that the central error handler returns a controlled 500 response.

8. Authentication, authorization, and security

Authentication establishes identity; authorization checks whether that identity may perform a particular action on a particular resource. Enforce authorization server-side on every protected request, including nested resources. Use TLS, secure cookie settings where cookies are used, careful CORS configuration, and rate limits appropriate to the endpoint.

```
function requireRole(role) {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ error: 'unauthenticated' });
    }
    if (!req.user.roles?.includes(role)) {
      return res.status(403).json({ error: 'forbidden' });
    }
    next();
  };
}

router.delete('/:id', authenticate, requireRole('admin'),
              asyncRoute(deleteBook));
```

Design notes: Never trust a user ID supplied by the client as proof of ownership. Use parameterized database operations, avoid unsafe dynamic evaluation, limit payload size, protect secrets, and keep dependencies updated. CORS is not authentication and does not prevent non-browser clients from calling an API.

Practice: Threat-model an endpoint that fetches an invoice by ID. What prevents one customer from reading another customer's invoice?

Additional review: Optimization lens: measure behavior with representative data and workload; check the relevant official documentation before relying on implementation-specific behavior.

9. Pagination, filtering, and idempotency

Unbounded list endpoints can become expensive and unstable. Offset pagination is easy to implement but can become slow at large offsets and can shift as rows change. Cursor pagination can use a stable, indexed sort key. Define maximum page sizes and deterministic ordering.

```
// Simple offset pagination; validate values before querying.
const page = Math.max(1, Number(req.query.page ?? 1));
const limit = Math.min(100, Math.max(1, Number(req.query.limit ?? 20)));
const result = await repository.list({
  offset: (page - 1) * limit,
  limit,
  orderBy: 'createdAt_desc'
});
res.json({
  data: result.items,
  page,
  limit,
  total: result.total
});
```

Design notes: For create operations that clients may retry after a timeout, consider idempotency keys backed by durable storage. Define duplicate behavior, sort order, and whether pagination totals are exact or approximate.

Practice: Compare page-based and cursor-based contracts for a feed where new records arrive continuously.

10. Testing, observability, and deployment

Test behavior at the HTTP boundary: status, headers, response body, validation, authentication, and error cases. Unit tests cover domain logic; integration tests exercise middleware and persistence; contract tests protect clients from breaking changes. Logs should be structured and should avoid tokens, passwords, and sensitive personal data.

```
// Example test shape using a request-testing library:
it('rejects a book with an empty title', async () => {
  const response = await request(app)
    .post('/api/v1/books')
    .send({ title: '   ', author: 'A. Writer' });

  expect(response.status).toBe(400);
  expect(response.body.error).toBe('invalid_title');
});
```

Design notes: Export the app separately from the listener to simplify tests. In production, use health checks, graceful shutdown, timeouts, dependency monitoring, and a documented deployment configuration. Track latency percentiles and error rates, not just raw request counts.

Practice: Capstone: implement CRUD for books with validation, a repository abstraction, auth on mutations, central errors, pagination, and tests for success and failure paths.

Trusted documentation and further reading

- **Node.js — Learn** — https://nodejs.org/en/learn
- **Node.js — HTTP module** — https://nodejs.org/api/http.html
- **Express — Routing** — https://expressjs.com/en/guide/routing.html
- **Express — Using middleware** — https://expressjs.com/en/guide/using-middleware.html
- **Express — Error handling** — https://expressjs.com/en/guide/error-handling.html
- **Express — Production best practices: security** — https://expressjs.com/en/advanced/best-practice-security.html
- **Express — Writing middleware** — https://expressjs.com/en/guide/writing-middleware.html
- **OWASP — REST Security Cheat Sheet** — https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

Subject review checklist

- HTTP method and status semantics are consistent.
- Middleware order and async error paths are explicit.
- Inputs are validated and authorization is server-side.
- List endpoints are bounded and errors do not leak internals.
- Success and failure paths have automated tests.